Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 1

по курсу «Алгоритмы компьютерной графики»

«Геометрическое преобразование графического примитива»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

1. Реализовать любой графический примитив.
2. Добавить любое геометрическое преобразование.
3. Добавить обработку события.

2 Основная теория

1. Реализация графического примитива

В качестве графических примитивов в программе используются два многоугольника (GL_POLYGON). Это один из базовых примитивов OpenGL, который позволяет отобразить закрашенную фигуру, заданную последовательностью вершин.

Первый примитив состоит из четырех вершин, описывающих неправильный четырехугольник. Для каждой вершины задан свой уникальный цвет с помощью команды glColor3f(), что создает эффект градиентной заливки. Координаты вершин задаются относительно переменных posx и posy, что в дальнейшем позволяет управлять их положением.

Второй примитив также является четырехугольником, однако порядок задания его вершин отличается от стандартного обхода по или против часовой стрелки. В контексте использования GL_POLYGON это может приводить к построению невыпуклого или самопересекающегося многоугольника. Поскольку спецификация фиксированного конвейера OpenGL корректно определяет поведение в первую очередь для простых выпуклых многоугольников, результат растеризации при нарушении порядка вершин может быть неопределённым и зависеть от реализации драйвера. Данный приём демонстрирует влияние порядка обхода вершин на итоговую геометрию примитива.

Для управления цветовой гаммой всей сцены используется глобальная переменная color. Она добавляется к каждому из трех компонентов цвета (R, G, B), что позволяет плавно изменять оттенок всех вершин одновременно, сохраняя при этом исходные цветовые соотношения между ними.

2. Добавление геометрического преобразования

В программе реализовано вращение всей сцены как единого целого. Это достигается за счет использования матричных операций OpenGL:

Матрица моделирования-вида: Перед отрисовкой примитивов вызывается функция $\mathrm{glLoadIdentity}()$, которая сбрасывает текущую матрицу в единичную, обеспечивая предсказуемость последующих преобразований.

Стек матриц: Вызов $\mathrm{glPushMatrix}()$ сохраняет текущее состояние матрицы, чтобы изолировать последующие трансформации от возможных изменений в других частях потенциально более сложного кода.

Применение поворота: Функция $\mathrm{glRotatef}(\mathrm{angle},\ 0,\ 0,\ 1)$ применяет к текущей матрице преобразование поворота. Примитивы, отрисованные после этого вызова, будут повернуты на угол angle вокруг оси Z (перпендикулярной экрану).

$$R = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Анимация: В конце каждой итерации главного цикла значение переменной angle увеличивается на фиксированную величину delta. Это создает эффект непрерывного вращения объектов.

После выполнения всех команд отрисовки вызывается $\mathrm{glPopMatrix}()$, который восстанавливает матрицу, сохраненную перед началом преобразований.

3. Добавление обработки событий

Приложение активно взаимодействует с пользователем через два типа событий, обрабатываемых с помощью колбэков библиотеки GLFW: нажатие клавиш и прокрутка колеса мыши.

- **Обработка нажатий клавиш (**$\mathrm{key_callback}$**):**
  - Программа отслеживает нажатие клавиш со стрелками: $\mathrm{KEY_RIGHT}$ и $\mathrm{KEY_LEFT}$.

- При каждом нажатии изменяется глобальная переменная magic_num: увеличивается на 0.03 при нажатии правой стрелки и уменьшается при нажатии левой.
- Переменная magic_num влияет на координаты некоторых вершин (например, 0.25 - magic_num для третьей вершины первого полигона и -0.25 + magic_num для первой вершины второго полигона). Таким образом, можно деформировать форму полигонов, сдвигая отдельные вершины навстречу друг другу или в противоположные стороны.

- **Обработка прокрутки колесика мыши (**scroll_callback**):**
  - Функция реагирует на прокрутку, используя параметр yoffset, который содержит величину и направление прокрутки. Направление изменения цвета определяется анализом параметра xoffset: при положительном смещении цвет изменяется в одну сторону, при отрицательном или нулевом - в другую.
  - В зависимости от условия, значение глобальной переменной color либо увеличивается, либо уменьшается на величину yoffset/100. Это позволяет плавно изменять цветовую гамму, прокручивая колесико мыши вверх или вниз.

3 Практическая реализация

Листинг 1: Основной код

```
import glfw
from OpenGL.GL import *

delta = 0.2
angle = 0.0
posx = 0.0
posy = 0.0
color = 0.0
magic_num = 0.0
flag = True

```

```
def main():
    if not glfw.init():
        return
    window = glfw.create_window(640, 640, "Lab1", None, None)
    if not window:
        glfw.terminate()
        return
    glfw.make_context_current(window)
    glfw.set_key_callback(window, key_callback)
    glfw.set_scroll_callback(window, scroll_callback)
    while not glfw.window_should_close(window):
        display(window)
    glfw.destroy_window(window)
    glfw.terminate()


def display(window):
    global angle
    global magic_num
    global color
    glClear(GL_COLOR_BUFFER_BIT)
    glLoadIdentity()
    glClearColor(1.0, 1.0, 1.0, 1.0)
    glPushMatrix()
    glRotatef(angle, 0, 0, 1)
    glBegin(GL_POLYGON)
    glColor3f(0.0 + color,0.95 + color,0.6 + color)
    glVertex2f(posx + 0.5,posy + 0.5)
    glColor3f(0.96 + color,0.6 + color,0.0 + color)
    glVertex2f(posx + -0.5,posy + 0.5)
    glColor3f(0.32 + color,0.1 + color,0.71 + color)
    glVertex2f(posx + 0.25 - magic_num,posy + 0.25 - magic_num)
    glColor3f(0.13 + color,0.32 + color,0.65 + color)
    glVertex2f(posx + 0.5,posy + -0.5)
    glEnd()

    glBegin(GL_POLYGON)
    glColor3f(0.4 + color,0.6 + color,0.6 + color)
    glVertex2f(posx - 0.25 + magic_num,posy - 0.25 + magic_num)
    glColor3f(0. + color,0.0 + color,0.89 + color)
    glVertex2f(posx + -0.5,posy + 0.5)
    glColor3f(0.45 + color,0.66 + color,0.49 + color)
    glVertex2f(posx - 0.5,posy - 0.5)
    glColor3f(0.78 + color,0.23 + color, 0.89 + color)
    glVertex2f(posx + 0.5,posy + -0.5)
    glEnd()
```

```
58
59      glPopMatrix()
60      angle += delta
61      glfw.swap_buffers(window)
62      glfw.poll_events()
63
64  def key_callback(window, key, scancode, action, mods):
65      global magic_num
66      global flag
67      if action == glfw.PRESS:
68          if key == glfw.KEY_RIGHT:
69              if flag:
70                  magic_num += 0.03
71          if key == 263: # glfw.KEY_LEFT
72              if flag:
73                  magic_num += -0.03
74
75  def scroll_callback(window, xoffset, yoffset):
76      global color
77      if (xoffset > 0):
78          color -= yoffset/100
79      else:
80          color += yoffset/100
81
82
83  if __name__ == "__main__":
84      main()
```

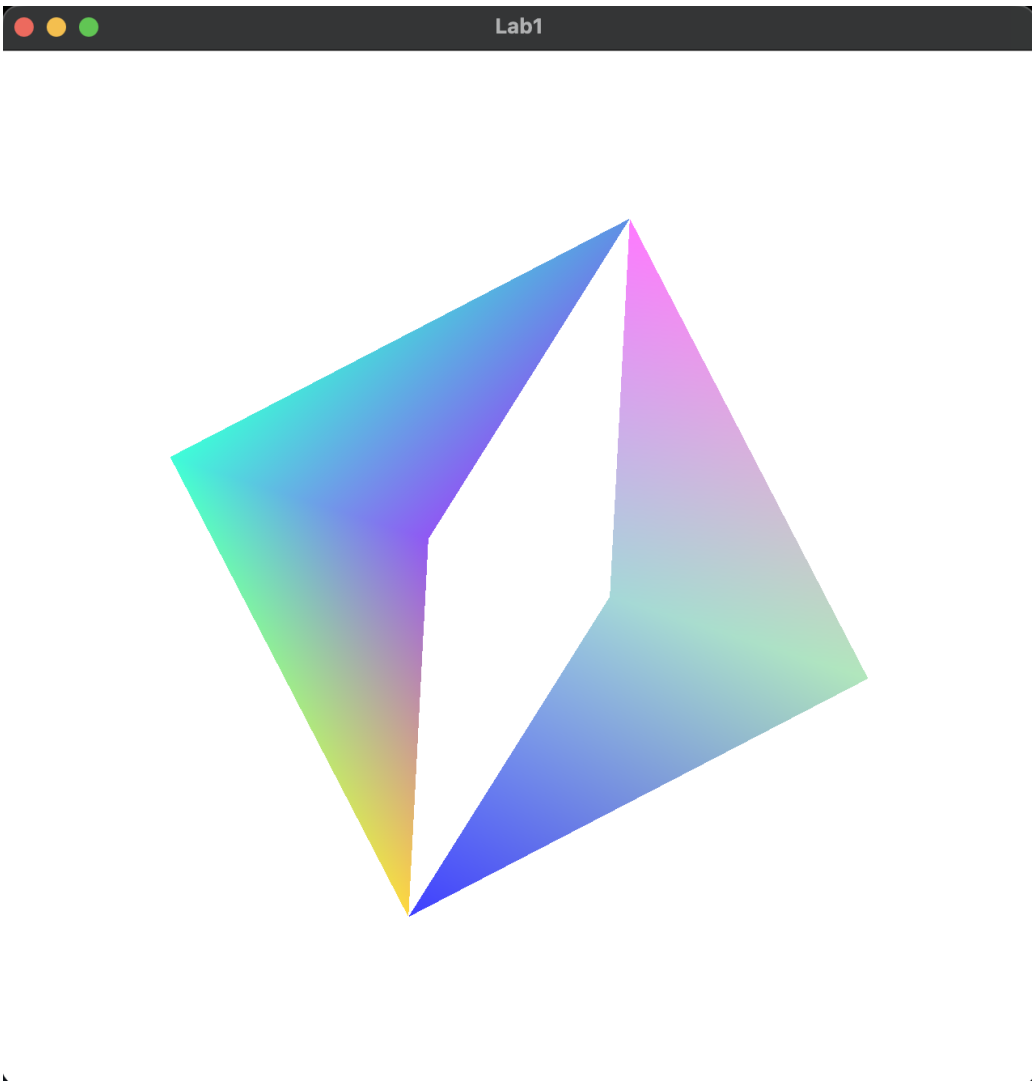


Рис. 1 — Результат работы

4 Вывод

В ходе лабораторной работы было разработано интерактивное приложение на Python с использованием OpenGL. Реализована отрисовка цветных многоугольников, к которым применено преобразование вращения. Добавлена обработка событий клавиатуры и мыши для изменения параметров объектов в реальном времени. Благодаря этой лабораторной работе я начал свой путь в изучении алгоритмов компьютерной графики и библиотеки OpenGL.